AW882XX Android Driver(MTK)

Version: V2.2

Date: Nov , 2023

contents

1. INFORMATION

Driver File	aw882xx_calib.c, aw882xx_calib.h, aw882xx_monitor.c, aw882xx_monitor.h, aw882xx.c, aw882xx.h, aw882xx_device.c, aw882xx_device.h, aw882xx_dsp.c, aw882xx_dsp.h, aw882xx_init.c, aw882xx_log,h, aw882xx_spin.c, aw882xx_spin.h, aw882xx_data_type.h, aw882xx_pid_1852_reg.h, aw882xx_bin_parse,c, aw882xx_bin_parse.h, aw882xx_pid_2013_reg.h, aw882xx_pid_2032_reg.h, aw882xx_pid_2055_reg.h, aw882xx_pid_2055a_reg.h, aw882xx_pid_2071_reg.h, aw882xx_pid_2113_reg.h, aw882xx_pid_2308_reg.h
Smart PA(with IV)	aw88257、aw88258、aw88261、aw88261s、aw88262、aw88263、aw88263h、aw88263s、aw88264、aw88265、aw88266、aw88266s、aw88270、aw88274、aw88299、aw88461
Smart PA(no IV)	aw88298、aw88298g、aw88266a、aw88230（No calibration function, please ignore the integration in Chapter 4/5）
I²C Address	0x30/0x31/0x32/0x33/0x34/0x35/0x36/0x37
Platform	mt6853

2. DRIVER PORTING

2.1 SMARTPA CONFIGURATION

2.1.1 Add PA Configuration

Add aw882xx smartpa in ProjectXXX.mk

```
MTK_AUDIO_SPEAKER_PATH = smartpa_awinic_aw882xx
```

Configuring the above options will be displayed The following parameters when Overall compilation in AudioParamOptions.xml

```
<Param name="MTK_AUDIO_SPEAKER_PATH" value="smartpa_awinic_aw882xx" />
```

Add Configuration in SmartPa_AudioParam.xml

```
--- a/audio_param/SmartPa_AudioParam.xml
+++ b/audio_param/SmartPa_AudioParam.xml
@@ -8,6 +8,7 @@
    <Param path="smartpa_cirrus_cs35l35" param_id="4"/>
    <Param path="smartpa_mtk_dummy" param_id="5"/>
    <Param path="smartpa_mtk_mt6660" param_id="5"/>
+   <Param path="smartpa_awinic_aw882xx" param_id="20"/>
  </ParamTree>
  <ParamUnitPool>
    <ParamUnit param_id="0">
@@ -76,5 +77,16 @@
      <Param name="is_apll_needed" value="1"/>
      <Param name="i2s_set_stage" value="4"/>
    </ParamUnit>
```



```
+    <ParamUnit param_id="20">
+      <Param name="have_dsp" value="2"/>
+      <Param name="is_alsa_codec" value="1"/>
+      <Param name="chip_delay_us" value="22000"/>
+      <Param name="supported_rate_list"
 value="0x1F40,0x2B11,0x2EE0,0x3E80,0x5622,0x5DC0,0x7D00,0xAC44,0xBB80"/>
+      <Param name="spk_lib_path" value=""/>
+      <Param name="spk_lib64_path" value=""/>
+      <Param name="codec_ctl_name" value=""/>
+      <Param name="is_apll_needed" value="1"/>
+      <Param name="i2s_set_stage" value="5"/>
+    </ParamUnit>
   </ParamUnitPool>
 </AudioParam>
```

Add description of aw882xx in SmartPa_ParamUnitDesc.xml

```
<Category name="smartpa_awinic_aw882xx"/>
```

2.2 AW882XX DRIVER PORTING

2.2.1 DTS Configuration

Single PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
    i2c_x {                                 /*x means the i2c bus number*/
+         /* AWINIC AW882XX mono Smart PA */
+         aw882xx_smartpa_0: aw882xx_smartpa@34 {
+         compatible = "awinic,aw882xx_smartpa";
+         #sound-dai-cells = <0>;
+         reg = <0x34>;
+         /* You cannot configure reset-gpio for aw88230, aw88257, aw88261 or
aw88265*/
+         reset-gpio = <&pio 89 0>;
+         irq-gpio = <&pio 37 0x0>;
+         /*sync-load = <1>;*/    /* Firmware loading mode. If the driver is
compiled in KO mode, it needs to be configured as 1*/
+         aw-re-min = <4000>;           /*Minimum calibration value (mohms)*/
+         aw-re-max= <30000>;            /*Maximum calibration value (mohms)*/
+         /*aw-cali-mode = "none";*/
+         status = "okay";
+      };
+      /* AWINIC AW882XX mono Smart PA End */
       /*re means resistance of speaker */
```

Multiple PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
```



```
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
&i2c_x {                                    /*x means the i2c bus number*/
+       aw882xx_smartpa_0: aw882xx@34 {
+           compatible = "awinic,aw882xx_smartpa";
+           #sound-dai-cells = <0>;
+           reg = <0x34>;
+           reset-gpio = <&pio 89 0x0>;
+           irq-gpio = <&pio 37 0x0>;
+           /*sync-load = <1>;*/
+           sound-channel = <0>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+           aw-re-min = <4000>;
+           aw-re-max= <30000>;
+           /*aw-cali-mode = "none";*/
+           status = "okay";
+       };
+      aw882xx_smartpa_1: aw882xx@35 {
+           compatible = "awinic, aw882xx_smartpa";
+           #sound-dai-cells = <0>;
+           reg = <0x35>;
+           reset-gpio = <&pio 17 0x0>;
+           irq-gpio = <&pio 19 0x0>;
+           /*sync-load = <1>;*/
+           sound-channel = <1>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+           aw-re-min = <4000>;
+           aw-re-max= <30000>;
+           /*aw-cali-mode = "none";*/
+           status = "okay";
+      };
+};
```

2.2.2 Driver Configuration

Defconfig Configuration

```
#add aw882xx smartpa
CONFIG_SND_SMARTPA_AW882XX = y
```

Create aw882xx directory in the kernel codecs directory and add driver files

(If the MTK platform has no DSP, delete the AW_MTK_PLATFORM_WITH_DSP macro definition in aw882xx_dsp.h)

```
aw882xx_calib.c, aw882xx_calib.h, aw882xx_monitor.c, aw882xx_monitor.h,
aw882xx.c, aw882xx.h, aw882xx_device.c, aw882xx_device.h, aw882xx_dsp.c,
aw882xx_dsp.h, aw882xx_init.c, aw882xx_log,h, aw882xx_spin.c,
aw882xx_spin.h, aw882xx_data_type.h, aw882xx_pid_1852_reg.h,
aw882xx_bin_parse,c, aw882xx_bin_parse.h, aw882xx_pid_2013_reg.h,
aw882xx_pid_2032_reg.h, aw882xx_pid_2055_reg.h, aw882xx_pid_2055a_reg.h,
aw882xx_pid_2071_reg.h, aw882xx_pid_2113_reg.h, aw882xx_pid_2308_reg.h
```

Kconfig Configuration

```
config SND_SMARTPA_AW882XX
    tristate "SoC Audio for awinic aw882xxseries"
    depends on I2C
    help
      This option enables support for aw882xxseries Smart PA.
```


Makefile Configuration

```
#for AWINIC AW882XX Smart PA
obj-$(CONFIG_SND_SMARTPA_AW882XX) += aw882xx/aw882xx.o
aw882xx/aw882xx_monitor.o aw882xx/aw882xx_init.o aw882xx/aw882xx_dsp.o
aw882xx/aw882xx_device.o aw882xx/aw882xx_calib.o
aw882xx/aw882xx_bin_parse.o aw882xx/aw882xx_spin.o
```

2.2.3 BIN Configuration

PA needs to configure register parameters to work normally. The configuration steps of PA bin file are as follows:

Compile Configuration

Add the bin file compilation option at the corresponding location of the platform

```
PRODUCT_COPY_FILES += \
xxxx/aw882xx_acf.bin:$(TARGET_COPY_OUT_VENDOR)/firmware/aw882xx_acf.bin

/*xxxx means Platform path*/
```

Path Configuration

Add the directory of bin file in the firmware_class.c, the general directory is vendor/firmware

```
static const char * const fw_path[] = {
    fw_path_para,
    "/vendor/firmware",              /*Add a path*/
    "/system/etc/firmware",
    "/lib/firmware/updates/" UTS_RELEASE,
    "/lib/firmware/updates",
    "/lib/firmware/" UTS_RELEASE,
    "/lib/firmware"
};
```

(PS：In the debugging stage, you can directly push the bin file to /vendor/ firmware/)

Bin Selection

Select the bin file **in the config directory of the driver package** based on the platform signal output format, PA quantity, and product name.

2.3 Platform Driver Configuration

2.3.1 DAI_LINK Configuration

Different platform dai_link configuration is as follows:

4G Platform Configuration

1) Add awinic_codecs array

Single PA Configuration

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-34",        /*I2C BUS:6, I2C Address:34*/
        .name = "aw882xx_smartpa.6-0034",
    },
};
```

Multiple PA Configuration

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-34",       /*I2C BUS:6, I2C Address:34*/
        .name = "aw882xx_smartpa.6-0034",
    },
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-6-35",       /*I2C BUS:6, I2C Address:35*/
        .name = "aw882xx_smartpa.6-0035",
    },
};
/*The above example is the dual PA configuration. When there are multiple PAS,
the configuration is added in turn*/
```

2) Modify DAI interface

```
static struct snd_soc_dai_link mt_soc_extspk_dai[] = {
    {
        .name = "Ext_Speaker_Multimedia",
        .stream_name = MT_SOC_SPEAKER_STREAM_NAME,
        .cpu_dai_name = "snd-soc-dummy-dai",
        .platform_name = "snd-soc-dummy",
    +#ifdef CONFIG_SND_SMARTPA_AW882XX
    +   .num_codecs = ARRAY_SIZE(awinic_codecs),
    +   .codecs = awinic_codecs,
    +#endif
        .ops = &mt_machine_audio_ops,
    }
}
```

5G Platform Configuration

If the kernel version is earlier than 5.4, perform the following operations：

Single PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm64/boot/dts/mediatek/mt6853.dts
+++ b/arch/arm/boot/dts/mediatek/mt6853.dts
@@ -2824,7 +2824,7 @@
        mtk_spk_i2s_in = <0>;
        /* mtk_spk_i2s_mck = <3>; */
        mediatek,speaker-codec {
-           sound-dai = <&speaker_amp>;
+           sound-dai = <&aw882xx_smartpa_0>;
        };
```



```
    };
 /* Add Dai information according to DTS node <aw882xx_smartpa_0> */
```

Multiple PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm64/boot/dts/mediatek/mt6853.dts
+++ b/arch/arm/boot/dts/mediatek/mt6853.dts
@@ -2824,7 +2824,7 @@
        mtk_spk_i2s_in = <0>;
        /* mtk_spk_i2s_mck = <3>; */
        mediatek,speaker-codec {
-           sound-dai = <&speaker_amp>;
+           sound-dai = <&aw882xx_smartpa_0 &aw882xx_smartpa_1>;
        };
    };
 /* Take two PA as an example, where the information corresponding to I2C node
is <aw882xx_smartpa_0>、<aw882xx_smartpa_1>*/
```

If the kernel version is later than 5.4, perform the following operations

Mono PA Configuration

```
/* Refer to the file path: kernel_path/sound/soc/mt6789/mt6789_mt6366.c */
/* i2c0 */
SND_SOC_DAILINK_DEFS(i2s0,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S0")),
    /*Assume that the i2c bus number is 6 and the device address is 0x34 */
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
/* i2s3 */
SND_SOC_DAILINK_DEFS(i2s3,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S3")),
    /* Assume that the i2c bus number is 6 and the device address is 0x34 */
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
```

Multi PA configuration

```
/* Refer to the file path:kernel_path/sound/soc/mt6789/mt6789_mt6366.c */
/* i2c0 */
SND_SOC_DAILINK_DEFS(i2s0,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S0")),
    /* Assume that the i2c bus number is 6 and the device address is 0x34 and 0x35 */
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34"),
                       COMP_CODEC("aw882xx_smartpa.6-0035", "aw882xx-aif-6-35")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
/* i2s3 */
SND_SOC_DAILINK_DEFS(i2s3,
    DAILINK_COMP_ARRAY(COMP_CPU("I2S3")),
    /* Assume that the i2c bus number is 6 and the device address is 0x34 and 0x35 */
    DAILINK_COMP_ARRAY(COMP_CODEC("aw882xx_smartpa.6-0034", "aw882xx-aif-6-34"),
                       COMP_CODEC("aw882xx_smartpa.6-0035", "aw882xx-aif-6-35")),
    DAILINK_COMP_ARRAY(COMP_EMPTY()));
```

2.4 CONFIGURATION I2S ROUTES(ONLY FOR 5G PLATFORM)

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
    sound: sound {
        compatible = "mediatek,mt6853-mt6359-sound";
        mediatek,audio-codec = <&mt6359_snd>;
        mediatek,platform = <&afe>;
+       mtk_spk_i2s_out = <3>;
+       mtk_spk_i2s_in = <9>;
        mtk_spk_i2s_mck = <3>;
    };
```

2.5 DRIVER PORTING VERIFICATION

The above operations have completed the driver integration. Confirm that is effective through the following driver log:

I2C communication succeeded:

```
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: dai name [aw882xx-aif-6-34]
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: pstream_name name [Speaker_Playback-6-34]
[Awinic][6-0034]aw882xx_dai_drv_append_suffix: cstream_name name [Speaker_Capture-6-34]
[Awinic][6-0034]aw882xx_i2c_probe: dev_cnt 1
```

Sound card registration succeeded：

```
[Awinic][6-0034]aw882xx_codec_probe: enter
[Awinic][6-0034]aw882xx_add_codec_controls: enter
[Awinic][6-0034]aw882xx_request_firmware: load [aw882xx_acf.bin] , file size: [2016]
[Awinic]aw_dev_parse_check_acf_by_hdr: project name [A2113]
[Awinic]aw_dev_parse_check_acf_by_hdr: custom name [Awinic]
```

PA Bin loaded successfully：

```
[Awinic][6-0034]aw_monitor_parse_vol_data_v_0_1_1: ===parse vol end ===
[Awinic][6-0034]aw_dev_parse_skt_type: enter
[Awinic][6-0034]aw_dev_parse_skt_type: get dsp data prof cnt is 0
[Awinic][6-0034]aw_dev_parse_get_vaild_prof: get vaild profile:2
[Awinic][6-0034]aw_dev_parse_acf: parse cfg success
[Awinic][6-0034]aw_dev_soft_reset: soft reset done
[Awinic][6-0034]aw_dev_reg_fw_update: amppd_st=0x0000
```

Play music and PA makes sound：

```
[Awinic][6-0034]aw_dev_set_intmask: done
[Awinic][6-0034]aw_monitor_start: enter
[Awinic][6-0034]aw_check_bop_status: enter
[Awinic][6-0034]aw_check_bop_status: check done! bop status is 0
[Awinic][6-0034]aw_device_start: done
[Awinic][6-0034]aw882xx_start_pa: start success
```

3. ALGORITHM-PORTING

Please refer to the algorithm integration document provided by Awinic for integration.

3.1 ALGORITHM AUTHENTICATION FUNCTION

If there is regular white noise playback during PA operation (audio source 10s ->white noise 3s->audio source 10s ->white noise 3s->……)

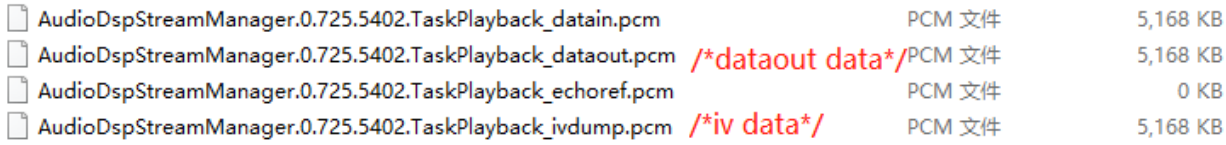
The above phenomenon indicates that the awinic algorithm authentication has failed. Please enable the corresponding function in the driver to recompile.

```
diff --git a/aw882xx_dsp.h b/aw882xx_dsp.h
index 998fa55..bce2a75 100644
--- a/aw882xx_dsp.h
+++ b/aw882xx_dsp.h
@@ -19,7 +19,7 @@

-/*#define AW_ALGO_AUTH_DSP*/
+#define AW_ALGO_AUTH_DSP

 /*factor form 12bit(4096) to 1000*/
 #define AW_DSP_RE_TO_SHOW_RE(re)    (((re) * (1000)) >> (12)
```

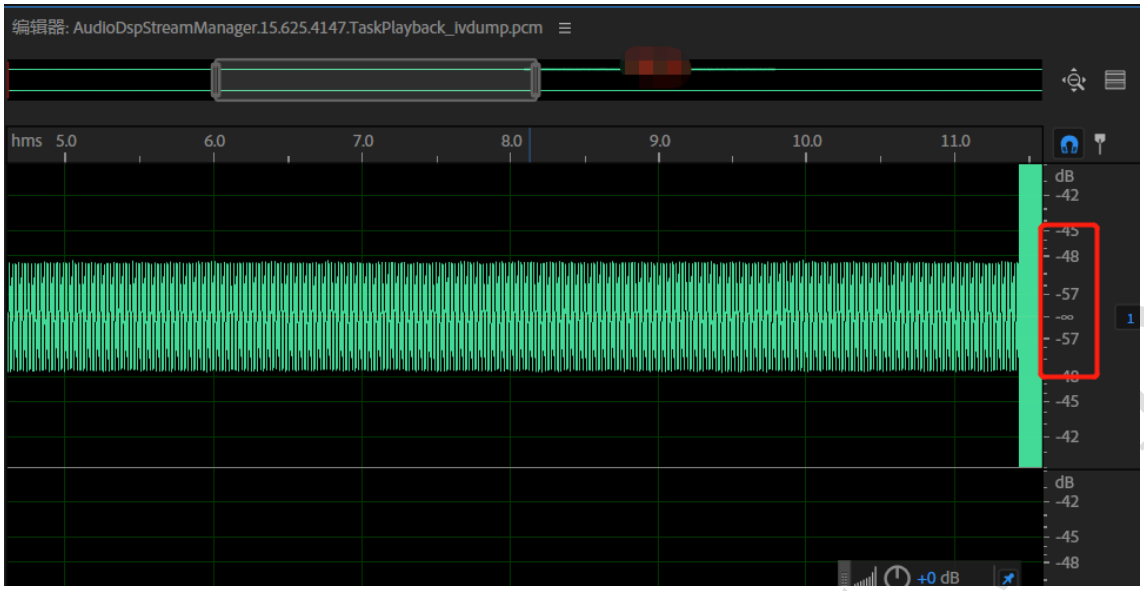
4. IV FEEDBACK FUNCTION VERIFICATION

Dump ADSP pcm data, please open and confirm whether the iv data is correct.

The naming rules of dataout and ivdump files are as follows:



By comparing dataout data and iv data, if the playback content is consistent, the loudness of iv data will be slightly lower, indicating that the iv feedback function has taken effect.



5. CALIBRATION

5.1 CALIBRATION PURPOSES

To meet speaker protection requirements, the AW882XX driver supports speaker calibration on the production line and writes the re value of a qualified speaker to the persist partition of the phone. When loading the chip configuration at boot, the driver will write the calibration value read in the persist partition into the protection algorithm to achieve the function of horn protection.

5.2 CALIBRATION ADAPTATION

5.2.1 Calibration File Saving Path Adaptation

The path to save the calibration value file is defined in driver (aw882xx_calib.c) as follows:

```
#ifdef AW_CALI_STORE_EXAMPLE
/*write cali to persist file example*/
#define AWINIC_CALI_FILE  "/mnt/vendor/persist/factory/audio/aw_cali.bin"  /*Save path*/
#define AW_INT_DEC_DIGIT 10
```

Please confirm whether there is the same path in the mobile phone. If there is no corresponding path, the calibration will fail:

```
msm8996:/ # cd mnt/vendor/persist/factory/audio/
msm8996:/mnt/vendor/persist/factory/audio # pwd
/mnt/vendor/persist/factory/audio
msm8996:/mnt/vendor/persist/factory/audio # ls
aw_cali.bin
msm8996:/mnt/vendor/persist/factory/audio #
```

5.3 CALIBRATION METHOD

AW882XX driver provides misc, class and attr calibration methods.

5.3.1 Class Calibration

1) Node function

Nodes	Fcuntion
/sys/class/smartpa/cali_time	1.Set calibration time 2.Show current calibration time
/sys/class/smartpa/re25_calib	1.Calibrate re 2.Save calibration re value
/sys/class/smartpa/f0_calib	Calibrate f0
/sys/class/smartpa/re_show	Show re
/sys/class/smartpa/f0_show	Show f0
/sys/class/smartpa/re_range	Show the range of calibration re value

2) Calibration steps

a. Play mute music；

b. Start the calibration：

```
msm8996:/sys/class/smartpa #
msm8996:/sys/class/smartpa # cat re25_calib
pri_l:6428 mOhms pri_r:6280 mOhms
msm8996:/sys/class/smartpa #
```

c. F0 calibration:

```
msm8996:/sys/class/smartpa #
msm8996:/sys/class/smartpa # cat f0_calib
pri_l:832 pri_r:980
msm8996:/sys/class/smartpa #
```

5.3.2 Misc Calibration

The misc calibration of AW882XX driver is realized through executable file. Follow the steps below:

1) Get the executable file

```
AW882XX_Driver_QCOM_v1.10.0          /*Driver root directory*/
   └ap
      └smartpa_cali
          │  aw882xx_cali_32        /*32bit system executable file*/
          │  aw882xx_cali_64        /*64bit system executable file*/
          │
          └example_source_code
                aw882xx_cali_attr_multi_mode.c
                aw882xx_cali_attr_single_dev_mode.c
                aw882xx_cali_class_example.c
```

2) Configure the executable file

```
C:\Users\AW882XX_Driver_MTK_v1.10.0\ap\smartpa_cali>adb push aw882xx_cali_32 /system/bin/
aw882xx_cali_32: 1 file pushed. 0.6 MB/s (34504 bytes in 0.052s)

C:\Users\AW882XX_Driver_MTK_v1.10.0\ap\smartpa_cali>adb shell chmod 0777 /system/bin/aw882xx_cali_32
```


3) Command introduction

```
Calibration executables version: v0.3.10
-------------------------------------------------------------
./aw882xx_cali [dev_name] cmd [optional params]        /*command format*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] cali [cali_re_time(ms)]        /*calibration*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] cali_re [cali_re_time(ms)]    /*re calibration*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] cali_f0 [noise]               /*f0 calibration*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] get_spkr_status          /*get realtime re\te\f0*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] get_spkr_st               /*get realtime re\te*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] set_cali_re re_value1 [re_value2]/*set cali_re value*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] cali_f0_q [noise]                /*cali f0\q*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] cali_all [cali_re_time(ms)]      /*cali re\f0\q*/
-------------------------------------------------------------
./aw882xx_cali [dev_name] get_re_range          /*get range of cali re value*/
-------------------------------------------------------------
./aw882xx_cali dev_name set_params  params_file      /*set params files to dsp*/
-------------------------------------------------------------
```

Parameter explanation (Note: [ ] means this option can be left blank)

dev_name	This parameter is used to calibrate a single device. The dev_name used by each device corresponds to the sound-channel configured in the DTS. **0: aw882xx_smartpa_l** **1: aw882xx_smartpa_r** **2: aw882xx_smartpa_sec_l** **3: aw882xx_smartpa_sec_r** If this parameter is not specified, all devices are calibrated by default.
noise	Fill in the noise parameter to play white noise when calibrating F0; If the customer chooses to play white noise by itself, you do not need to set this parameter.
cali_re_time	The time used to set the calibration RE value, in ms, shall not be less than 1000ms; If this parameter is not specified, the default calibration time is 3000ms.
re_value1	The calibration resistance value (mohm) needs to be set to the system.
params_file	Path corresponding to algorithm parameters.

4) Calibration steps

a. Play mute music;

b. Start the calibration:


```
msm8996:/ # aw882xx_cali_32 start_cali
[aw882xx_smartpa_1]cali_RE = 6429
[aw882xx_smartpa_r]cali_RE = 6346
[aw882xx_smartpa_1]cali_f0 = 836
[aw882xx_smartpa_r]cali_f0 = 979
```

5.3.3 Attr Calibration

1) Node path

6-0034: ”6”means I2C bus, ”0034”meand I2C address:

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # ls /*Node path*/
algo_ver cali_f0   dbg_prof dsp_re    modalias       name       print_dbg reg       uevent
awrw     cali_re   driver   f0_show   monitor        phase_sync re_range  rw
cali     cali_time drv_ver  fade_step monitor_update power      re_show   subsystem
```

2) Node function

Node	Function
cali_time	1.Set calibration time; 2.Show current calibration time.
cali	Turn on re and f0 calibration
cali_re	1.Calibrate re; 2.Save calibration re value.
cali_f0	Calibrate f0;
re_show	Show re.
f0_show	Show f0.
re_range	Show the range of calibration re value.

3) Calibration steps

a. Play mute music；

b. Start the calibration：

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_1:6371 mOhms pri_r:6380 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

c. F0 calibration:

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_f0
pri_1:830 pri_r:980
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

5.4 VERIFICATION OF CALIBRATION RESULTS

1) Calibrate several times to confirm whether the calibration re is within the effective range and the value will change slightly. (re effective range confirmed with hardware colleagues)，Calibration twice in attr mode, for example:

```
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_l:7177 mOhms pri_r:6742 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 # cat cali_re
pri_l:7157 mOhms pri_r:6737 mOhms
msm8996:/sys/bus/i2c/drivers/aw882xx_smartpa/6-0034 #
```

2) Check whether the re value is written to the file：

```
msm8996:/ # cat mnt/vendor/persist/factory/audio/aw_cali.bin
      7157      6737msm8996:/ #
msm8996:/ #
```

3) When playing music, check the dsp_re node, confirm that it is the same as the above calibration value：

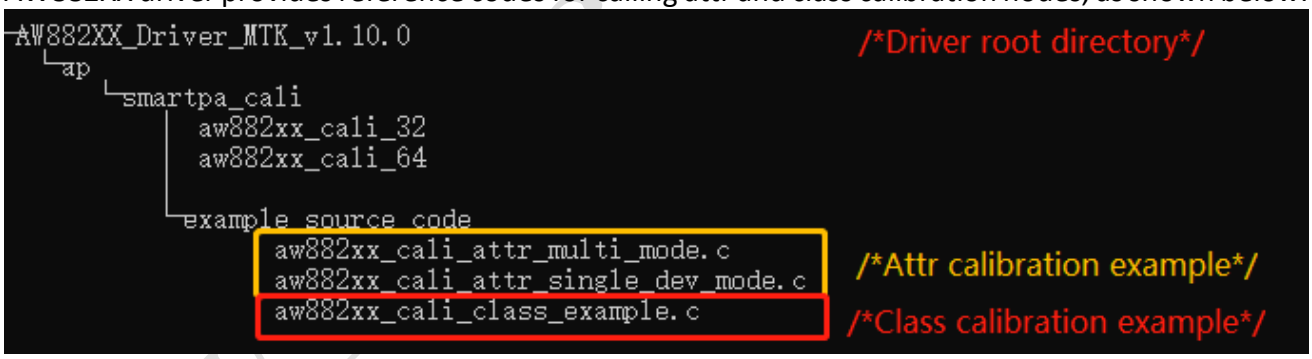
```
msm8996:/ # cat sys/bus/i2c/drivers/aw882xx_smartpa/6-0034/dsp_re
7156
msm8996:/ # cat sys/bus/i2c/drivers/aw882xx_smartpa/6-0037/dsp_re
6736
msm8996:/ #
```

(PS: A deviation of 1 in the read re value is normal)

4) Restart the phone, play music and check the DSP again_ Re node, confirm DSP_ The value in the re node is the same as the re value in the file。

5.5 CALIBRATION EXAMPLE CODE

AW882XX driver provides reference codes for calling attr and class calibration nodes, as shown below:



6. DEBUG INTERFACE

6.1 NODE

AW882XX driver will create multiple device node files with different functions. The path is sys/bus/i2c/drivers/aw882xx_smartpa/*-00xx, where * is the i2c bus number and xx is the i2c address. You can use adb to read and write nodes to debug AW882XX driver.

reg

Node name	reg
Description	read and write all register value of aw882xx

Instructions	read register value: cat reg write register value: echo reg_addr reg_data > reg (Hexadecimal operation)
Example	cat reg (get all the values of the register with read permission) echo 0x04 0x0241 > reg (write the value of 0x0241 to the register of 0x04)

rw

Node name	rw
Description	read and write single register value of aw882xx
Instructions	read register value: echo reg_addr > rw (Hexadecimal operation) cat rw write register value: echo reg_addr reg_data > rw (Hexadecimal operation)
Example	echo 0x04 > rw (read the value of the 0x04 register) cat rw echo 0x04 0x0241 > rw (write the value of 0x0241 to the register of 0x04)

driver_ver

Node name	driver_ver
Description	get driver version number
Instructions	get version number: cat driver_ver

dsp_re

Node name	dsp_re
Description	set or get the re value set in the algorithm
Instructions	get re: cat dsp_re set re: echo 7000 > dsp_re

fade_step

Node name	fade_step
Description	set step of fade in and fade out
Instructions	set step: echo step > fade_step get step: cat fade_step
Example	echo 6 > fade_step (set the step to 6) cat fade_step (get the current step of fade in and fade out)

dbg_prof

Node name	dbg_prof
Description	switch scene function control node
Instructions	switch scene function enable: echo 1 > dbg_prof switch scene function disable: echo 0 > dbg_prof get dbg_prof status: cat dbg_prof

phase_sync

Node name	phase_sync
Description	Phase synchronization function control node
Instructions	Phase synchronization function enable: echo 1 > phase_sync Phase synchronization function disable: echo 0 > phase_sync get phase_sync status: cat phase_sync

print_dbg

Node name	print_dbg
Description	i2c data printing control node
Instructions	i2c data printing enable: echo 1 > print_dbg i2c data printing disable: echo 0 > print_dbg get print_dbg status: cat print_dbg

algo_ver

Node name	algo_ver
Description	get the version of algo
Instructions	get the version of algo: cat algo_ver

monitor

Node name	monitor
Description	switch monitor function
Instructions	turn on the monitor function: echo 1 > monitor turn off the monitor function: echo 0 > monitor get monitor status: cat monitor

monitor_update

Node name	monitor_update
Description	update monitor.bin file
Instructions	update monitor.bin file: echo 1 > monitor_update

6.2 KCONTROL

Kcontrol	function	example
aw_dev_0_prof	mode selection	tinymix aw_dev_0_prof Music select Music mode tinymix aw_dev_0_prof Receiver select Receiver mode
aw_dev_0_switch	switch chip	tinymix aw_dev_0_switch Enable turn on the chip tinymix aw_dev_0_switch Disable turn off the chip

aw_dev_0_monitor	switch monitor	tinymix aw_dev_0_monitor Enable Enable monitor tinymix aw_dev_0_monitor Disable Disable monitor
aw882xx_rx_switch	switch mec	tinymix aw882xx_rx_switch 0 turn off mec tinymix aw882xx_rx_switch 1 turn on mec
aw882xx_tx_switch	switch tx	tinymix aw882xx_tx_switch 0 turn off tx tinymix aw882xx_tx_switch 1 turn on tx
aw882xx_spin_switch	set rotation angle	tinymix aw882xx_spin_switch spin_90 rotate 90 degrees tinymix aw882xx_spin_switch spin_180 rotate 180 degrees
aw882xx_fadein_us	set stepping time of fade in	tinymix aw882xx_fadein_us 500 set stepping time of fade in :500
aw882xx_fadeout_us	set stepping time of fade out	tinymix aw882xx_fadeout_us 500 set stepping time of fade out: 500

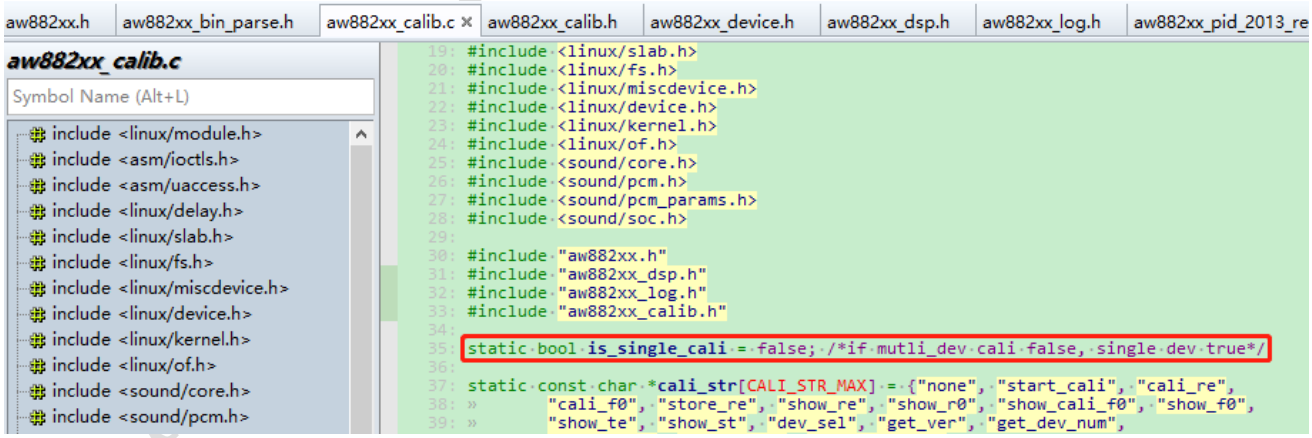
7. APPENDIX

7.1 ABOUT CALIBRATION

Awinic provides misc ioctl node calibration, device attr node calibration and class node calibration methods

Name	Description
/dev/awxx_smartpa	Support multi-PA calibration at the same time
/sys/class/smartpa	Support multi-PA calibration at the same time
/sys/bus/i2c/driver/aw882xx_smartpa/x-xx/	Support single PA or multiple PA calibration at the same time

The code defaults to misc/class multi-PA calibration at the same time. The device attr node can support both a single PA calibration and multiple PA calibrations. FAE or clients can configure the global variable `is_single_cali` in `aw882xx_calib.c` as needed.



If there is only a single PA, all nodes are single PA calibration.

7.2 PLATFORM I2C BUS DYNAMIC CHANGES

If the platform will change the i2c bus number, the i2c bus number of dynamic changes will cause the failure of dai_link matching, can solve by modifying the configuration in the device tree and dai_link configuration.

7.2.1 DTS Configuration

Add the rename-flag attribute to the driver node and set the attribute value to 1.

Single PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
    i2c_x {                                  /*x means the i2c bus number*/
+          /* AWINIC AW882XX mono Smart PA */
+          aw882xx_smartpa_0: aw882xx_smartpa@34 {
+          compatible = "awinic,aw882xx_smartpa";
+          #sound-dai-cells = <0>;
+          reg = <0x34>;
+          /* You cannot configure reset-gpio for aw88230, aw88257, aw88261 or
aw88265*/
+          reset-gpio = <&pio 89 0>;
+          irq-gpio = <&pio 37 0x0>;
+          aw-re-min = <4000>;           /*Minimum calibration value (mohms)*/
+          aw-re-max= <30000>;           /*Maximum calibration value (mohms)*/
+          /*aw-cali-mode = "none";*/
+          rename-flag = <1>;
+          status = "okay";
+      };
+      /* AWINIC AW882XX mono Smart PA End */
       /*re means resistance of speaker */
```

Multiple PA Configuration

```
diff --git a/arch/arm/boot/dts/mediatek/mt6853.dtsi
b/arch/arm/boot/dts/mediatek/mt6853.dtsi
index f22db2e..a340a32 100644
--- a/arch/arm/boot/dts/mediatek/mt6853.dtsi
+++ b/arch/arm/boot/dts/mediatek/mt6853.dtsi
@@ -549,6 +549,8 @@
&i2c_x {                                   /*x means the i2c bus number*/
+      aw882xx_smartpa_0: aw882xx@34 {
+          compatible = "awinic,aw882xx_smartpa";
+          #sound-dai-cells = <0>;
+          reg = <0x34>;
+          reset-gpio = <&pio 89 0x0>;
+          irq-gpio = <&pio 37 0x0>;
+          sound-channel = <0>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+          aw-re-min = <4000>;
+          aw-re-max= <30000>;
+          /*aw-cali-mode = "none";*/
+          rename-flag = <1>;
+          status = "okay";
+      };
+     aw882xx_smartpa_1: aw882xx@35 {
+          compatible = "awinic, aw882xx_smartpa";
+          #sound-dai-cells = <0>;
+          reg = <0x35>;
+          reset-gpio = <&pio 17 0x0>;
```



```
+           irq-gpio = <&pio 19 0x0>;
+           sound-channel = <1>; /*0:pri_l 1:pri_r 2:sec_l 3:sec_r*/
+           aw-re-min = <4000>;
+           aw-re-max= <30000>;
+           /*aw-cali-mode = "none";*/
+           rename-flag = <1>;
+           status = "okay";
+       };
+};
```

7.2.2 DAI_LINK Configuration

Change the suffix of codec_name and codec_dai_name to sound-channel,different platform dai_link configuration is as follows:

4G Platform Configuration

3) Add awinic_codecs array

Single PA Configuration

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-0",     /* Change the suffix to sound-channel of
DTS node */
        .name = "aw882xx_smartpa_0",
    },
};
```

Multiple PA Configuration

```
struct snd_soc_dai_link_component awinic_codecs[] = {
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-0",     /* Change the suffix to sound-channel of
DTS node */
        .name = "aw882xx_smartpa_0",
    },
    {
        .of_node = NULL,
        .dai_name = "aw882xx-aif-1",     /* Change the suffix to sound-channel of
DTS node */
        .name = "aw882xx_smartpa_1",
    },
};
/*The above example is the dual PA configuration. When there are multiple PAS,
the configuration is added in turn*/
```

4) Modify DAI interface

```
static struct snd_soc_dai_link mt_soc_extspk_dai[] = {
    {
        .name = "Ext_Speaker_Multimedia",
        .stream_name = MT_SOC_SPEAKER_STREAM_NAME,
        .cpu_dai_name = "snd-soc-dummy-dai",
        .platform_name = "snd-soc-dummy",
    +#ifdef CONFIG_SND_SMARTPA_AW882XX
    +   .num_codecs = ARRAY_SIZE(awinic_codecs),
    +   .codecs = awinic_codecs,
```



```
    +#endif
        .ops = &mt_machine_audio_ops,
    }
}
```

5G Platform Configuration

5G platforms need no additional modifications.

